

Entornos de Desarrollo.

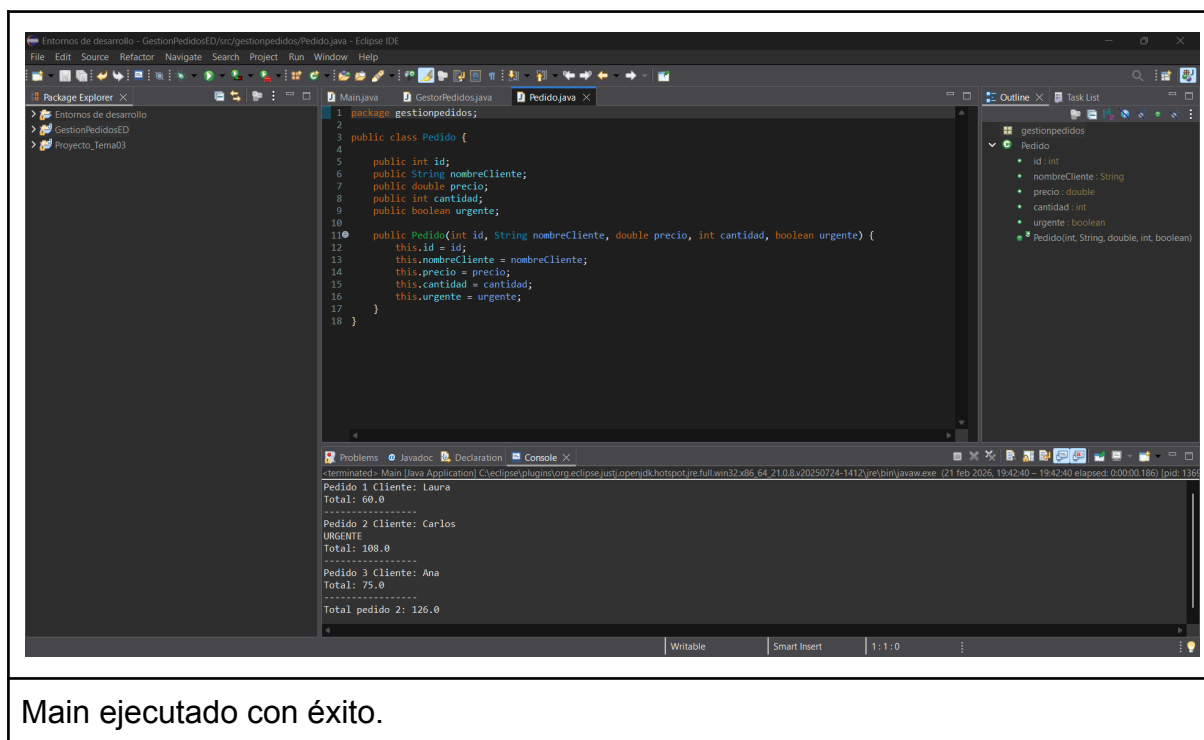
Tarea unidad 4.

Configuración inicial y primera ejecución del código sin cambios.....	2
Análisis y refactorización del código.....	3
Ejercicio 1. Análisis inicial del código.....	3
Ejercicio 2. Refactorización del proyecto.....	5
Control de versiones con Git.....	10
Ejercicio 3. Inicialización del repositorio.....	10
Ejercicio 4. Gestión de cambios.....	10
Documentación del código con JavaDoc.....	12
Ejercicio 5. Documentación de clases y métodos.....	12
Anexo.....	21
Prueba de trabajo personal:.....	21

Configuración inicial y primera ejecución del código sin cambios.

Tras crear un proyecto, configurarlo y copiar las tres clases que lo componen, ejecutamos el main proporcionado:

(NOTA: Si bien en la tarea se dice que el paquete debe ser `gestionpedidos`, el código proporcionado, que se nos indica que no debemos cambiar, establece las clases bajo el paquete `app`. Lo dejo en `gestionpedidos`.



Análisis y refactorización del código

Ejercicio 1. Análisis inicial del código

Clase	Método / Elemento	Comentario
Pedido	Diseño general	Clase sin comportamiento: La clase solo guarda datos. En este caso, el cálculo del precio del pedido debería estar dentro de esta clase, no en el gestor.
Pedido	Atributos de clase	Falta de encapsulamiento: Los atributos están como public. Deben ser private y protegerse su acceso mediante métodos getters/setters.
GestorPedidos	mostrarPedidos()	Mezcla de responsabilidades: Este método rompe el Principio de Responsabilidad Única. Hace la impresión en pantalla (presentación) y a la vez calcula descuentos (lógica de negocio).
GestorPedidos	mostrarPedidos() y calcularTotalPedido()	Código duplicado (Violación DRY): La fórmula de cálculo de descuentos y totales está escrita dos veces en diferentes lugares. Debería unificarse en un solo método (idealmente dentro de Pedido).
GestorPedidos	Atributo lista	Nombre genérico: El nombre lista no describe qué contiene. Es una buena práctica usar nombres en plural que den contexto, como pedidos.
GestorPedidos	añadirPedido()	Caracteres especiales en firmas: Usar la "ñ" es una mala práctica que genera problemas de codificación al cambiar de entorno. Es mejor usar agregarPedido.

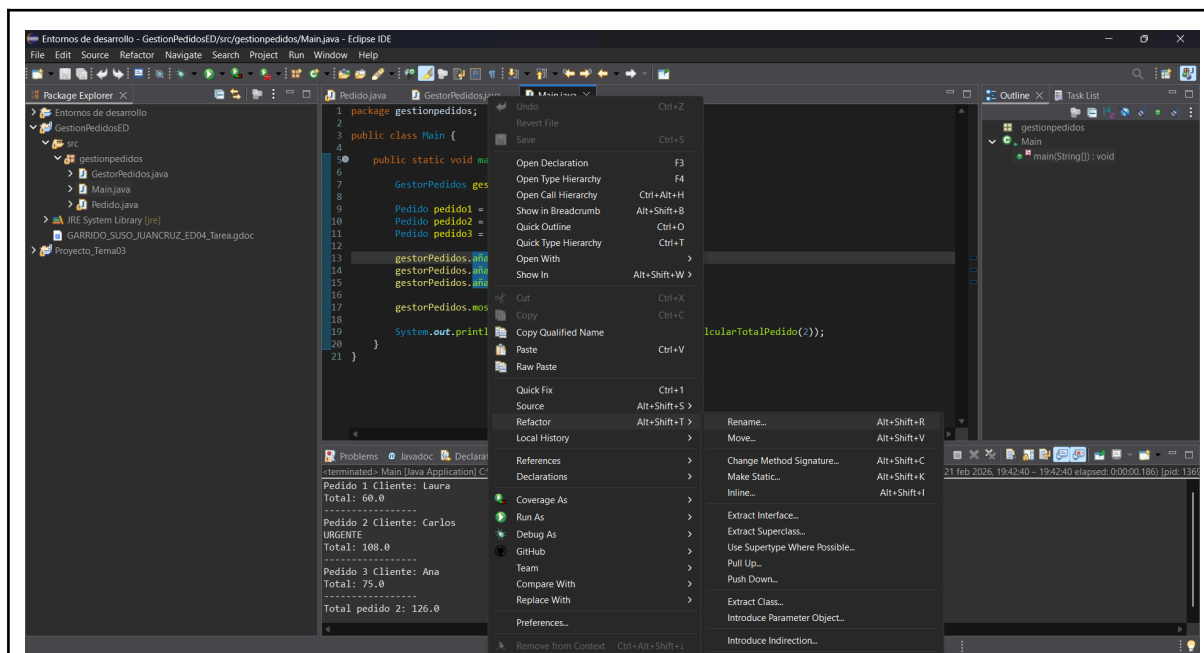
Main	main()	Variables poco descriptivas: Usar nombres de una letra como g, o p1, p2 resta legibilidad. Es mejor usar gestor, pedido1, etc.
GestorPedidos	Lógica de cálculo	Bug por inconsistencia: A causa de tener el código duplicado, en mostrarPedidos() se ha olvidado sumar los 20€ si el pedido es urgente. En calcularTotalPedido() sí se suman.
GestorPedidos	calcularTotalPedido()	Búsqueda ineficiente en Collection: Al recorrer la lista buscando un ID, no se interrumpe el bucle al encontrarlo. Falta un break o un return directo para evitar recorrer la lista entera innecesariamente. Por otro lado, si el pedido no existe, se devuelve cero sin indicar error.
GestorPedidos	mostrarPedidos()	Uso del bucle For: Se utiliza un bucle for tradicional con índice (i). Para recorrer Collections en Java de forma limpia es mejor usar el for-each (for (Pedido p : Collection)).
GestorPedidos	mostrarPedidos()	Comparación booleana redundante: La expresión if (p.urgente == true), basta con escribir if (p.urgente).
Pedido	Lógica de cálculo	Antipatrón de valores hardcoded: El recargo de urgencia (20) y el descuento (0.10) están escritos directamente en el código. Deben extraerse a constantes o a atributos para facilitar su mantenimiento futuro. Ignoro qué hacer si el hecho de que sea urgente implica entrar dentro del descuento, lo dejo en el orden que está.
GestorPedidos	Atributo lista	Elección de estructura de datos: Usar un List obliga a buscar los IDs con un bucle y se admiten ids repetidos. Es mucho más eficiente usar un Map<Integer, Pedido> (como HashMap o LinkedHashMap). Además con esto impedimos que dos pedidos tengan la misma ID, al violar en este caso el carácter de Set del conjunto de claves del Map.

Con estos comentarios, procedemos a la refactorización.

Ejercicio 2. Refactorización del proyecto

Comenzamos con la clase main, cambiaremos aquí los nombres de algunas variables y métodos con caracteres no recomendados, como añadirProducto.

Usamos las herramientas de eclipse de refactorización:



Procediendo al cambio de nombre del método, al hacerlo así, eclipse se encarga de cambiar la clase que declara el método y todas las que lo implementan y lo llaman

La clase nos queda así:

```
package gestionpedidos;

public class Main {

    public static void main(String[] args) {

        GestorPedidos gestorPedidos = new GestorPedidos();

        Pedido pedido1 = new Pedido(1, "Laura", 30, 2, false);
        Pedido pedido2 = new Pedido(2, "Carlos", 60, 2, true);
```

```

        Pedido pedido3 = new Pedido(3, "Ana", 15, 5, false);

        gestorPedidos.anyadirPedido(pedido1);
        gestorPedidos.anyadirPedido(pedido2);
        gestorPedidos.anyadirPedido(pedido3);

        gestorPedidos.mostrarPedidos();

        System.out.println("Total pedido 2: " +
        gestorPedidos.calcularTotalPedido(2));
    }
}

```

Ahora procederemos con los cambios en Pedido.

En pedido nos encontramos con tres aspectos:

- Encapsulación
- Responsabilidad de implementar el cálculo sobre su coste
- Eliminación de valores hardcodedos

Nuestro pedido solo está diseñado para una sola línea de pedido, es decir, un producto y una cantidad de él, más recargos y descuentos, implementaremos ese cálculo en Pedido, en el método calcularTotal()

La clase queda así:

```

package gestionpedidos;

public class Pedido {

    private int id;
    private String nombreCliente;
    private double precio;
    private int cantidad;
    private boolean urgente;
    private double costeUrgente;
    private double porcentajeDescuentoSobre100;

    public Pedido(int id, String nombreCliente, double precio, int cantidad,
    boolean urgente) {
        this.setId(id);
        this.setNombreCliente(nombreCliente);
    }
}

```

```

        this.setPrecio(precio);
        this.setCantidad(cantidad);
        this.setUrgente(urgente);
        this.setCosteUrgente(20);
        this.setPorcentajeDescuentoSobre100(0.1);
    }

    public double calcularTotal() {
        double total = getPrecio() * getCantidad();
        if (isUrgente()) {
            total = total + getCosteUrgente();
        }
        if (total > 100) {
            total = total - (total * getPorcentajeDescuentoSobre100());
        }
        return total;
    }

    public boolean isUrgente() {
        return urgente;
    }

    public void setUrgente(boolean urgente) {
        this.urgente = urgente;
    }

    public int getCantidad() {
        return cantidad;
    }

    public void setCantidad(int cantidad) {
        this.cantidad = cantidad;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public String getNombreCliente() {
        return nombreCliente;
    }

    public void setNombreCliente(String nombreCliente) {
        this.nombreCliente = nombreCliente;
    }

    public int getId() {

```

```

        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public double getPorcentajeDescuentoSobre100() {
        return porcentajeDescuentoSobre100;
    }

    public void setPorcentajeDescuentoSobre100(double
porcentajedescuentoSobre100) {
        this.porcentajeDescuentoSobre100 = porcentajedescuentoSobre100;
    }

    public double getCosteUrgente() {
        return costeUrgente;
    }

    public void setCosteUrgente(double costeUrgente) {
        this.costeUrgente = costeUrgente;
    }
}

```

Por último, acometemos los cambios en GestorPedidos.

En esta clase se ha delegado el cálculo del total al nuevo método centralizado en Pedido, se ha cambiado la List a Map (LinkedHashMap, que mantiene el orden de los valores), cambiado el nombre del atributo que guarda los pedidos y optimizado algún bucle, y se lanza un error (usando una excepción de java) cuando no se encuentra un id de pedido.

Código resultante:

```

package gestionpedidos;

import java.util.LinkedHashMap;
import java.util.Map;

public class GestorPedidos {

    private Map<Integer, Pedido> pedidos = new LinkedHashMap<Integer,
Pedido>();
}

```

```

public void anyadirPedido(Pedido p) {
    pedidos.put(p.getId(), p);
}

public void mostrarPedidos() {
    for (Pedido p : pedidos.values()) {
        System.out.println("Pedido " + p.getId() + " Cliente: " +
p.getNombreCliente());
        if (p.isUrgente()) {
            System.out.println("URGENTE");
        }
        System.out.println("Total: " + p.calcularTotal());
        System.out.println("-----");
    }
}

public double calcularTotalPedido(int idPedido) {
    Pedido pedido = pedidos.get(idPedido);
    if (pedido == null) {
        throw new IllegalArgumentException("No se encuentra el pedido con
ID: " + idPedido);
    }
    return pedido.calcularTotal();
}
}

```

Con estos cambios, ejecutamos el main, según muestro en el siguiente pantallazo.

```

Pedido 1 Cliente: Laura
Total: 60.0
Pedido 2 Cliente: Carlos
URGENTE
Total: 120.0
-----
Pedido 3 Cliente: Ana
Total: 75.0
-----
Total pedido 2: 120.0

```

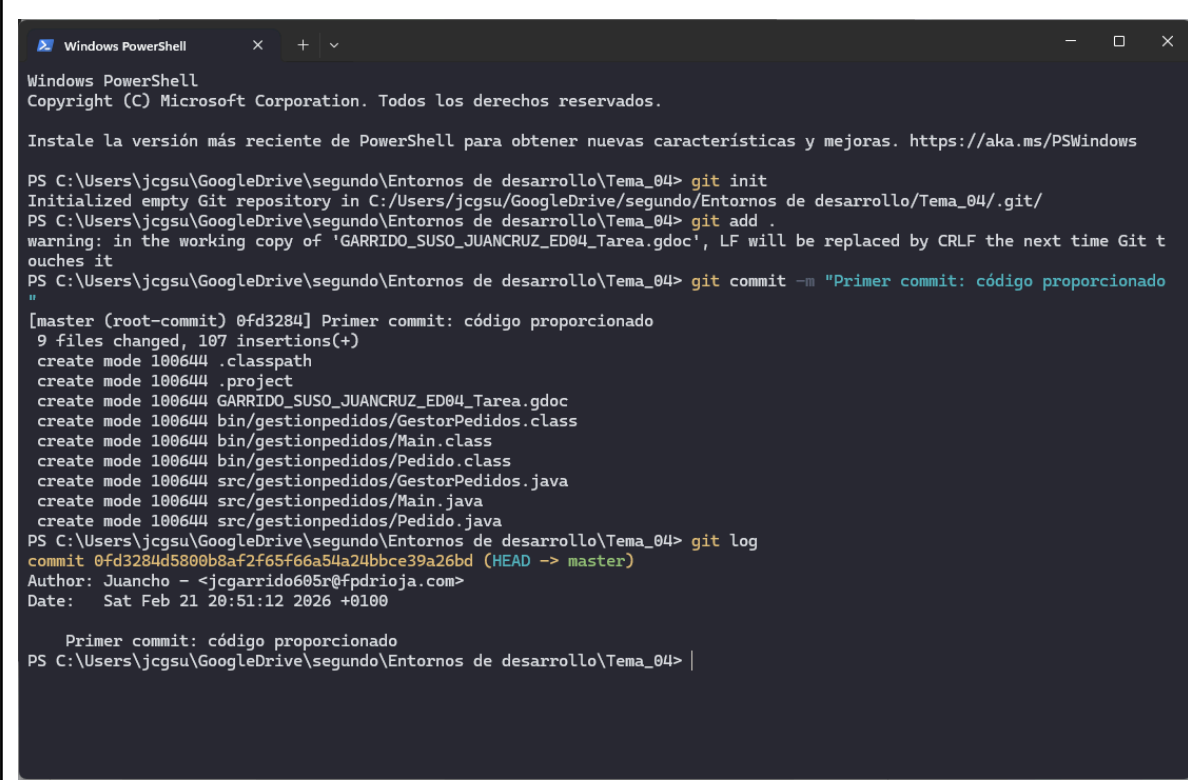
El funcionamiento ya es correcto para el pedido 2, lo demás queda igual que estaba.

Control de versiones con Git

Tras cada paso mostraré el git log del repositorio.

Ejercicio 3. Inicialización del repositorio

Iniciamos el repositorio, traemos a stage todo el contenido y hacemos el primer commit



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git init
Initialized empty Git repository in C:/Users/jcgsgu/GoogleDrive/segundo/Entornos de desarrollo/Tema_04/.git/
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git add .
warning: in the working copy of 'GARRIDO_SUSO_JUANCRUZ_ED04_Tarea.gdoc', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git commit -m "Primer commit: código proporcionado"
[master (root-commit) 0fd3284] Primer commit: código proporcionado
 9 files changed, 107 insertions(+)
 create mode 100644 .classpath
 create mode 100644 .project
 create mode 100644 GARRIDO_SUSO_JUANCRUZ_ED04_Tarea.gdoc
 create mode 100644 bin/gestionpedidos/GestorPedidos.class
 create mode 100644 bin/gestionpedidos/Main.class
 create mode 100644 bin/gestionpedidos/Pedido.class
 create mode 100644 src/gestionpedidos/GestorPedidos.java
 create mode 100644 src/gestionpedidos/Main.java
 create mode 100644 src/gestionpedidos/Pedido.java
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git log
commit 0fd3284d5800b8af2f65f66a54a24bbce39a26bd (HEAD -> master)
Author: Juancho - <jcgarrido605r@fpdrioja.com>
Date: Sat Feb 21 20:51:12 2026 +0100

    Primer commit: código proporcionado
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> |
```

Tareas realizadas con éxito. tras la creación inicial del proyecto

Ejercicio 4. Gestión de cambios

Ahora mostraremos cómo se hizo commit de la refactorización pedida.

```
Windows PowerShell
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git add .
warning: in the working copy of 'GARRIDO_SUSO_JUANCRUZ_ED04_Tarea.gdoc', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git commit -m "Refactorización realizada"
[master e35dceb] Refactorización realizada
 6 files changed, 115 insertions(+), 54 deletions(-)
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git log
commit e35dceb2f245bcb7fd99c3a2dd6f6570196dbf51d (HEAD -> master)
Author: Juancho - <jcgarrido605r@fpdrioja.com>
Date: Sat Feb 21 22:14:07 2026 +0100

    Refactorización realizada

commit 0fd3284d5800b8af2f65f66a54a24bbce39a26bd
Author: Juancho - <jcgarrido605r@fpdrioja.com>
Date: Sat Feb 21 20:51:12 2026 +0100

    Primer commit: código proporcionado
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> |
```

Tras haber implementado la refactorización, hacemos otro commit.

Por último, el control de versiones tras hacer la tarea sobre javadoc.

```
Windows PowerShell
Primer commit: código proporcionado
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git add .
warning: in the working copy of 'GARRIDO_SUSO_JUANCRUZ_ED04_Tarea.gdoc', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'doc/Legal/jquery.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'doc/Legal/jqueryUI.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'doc/script-dir/jquery-3.7.1.min.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'doc/script-dir/jquery-ui.min.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'doc/script-dir/jquery-ui.min.js', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git commit -m "Javadoc actualizado"
[master cfd3c32] Javadoc actualizado
50 files changed, 4855 insertions(+), 4 deletions(-)
create mode 100644 doc/allclasses-index.html
create mode 100644 doc/allpackages-index.html
create mode 100644 doc/copy.svg
create mode 100644 doc/element-list
create mode 100644 doc/gestionpedidos/GestorPedidos.html
create mode 100644 doc/gestionpedidos/Main.html
create mode 100644 doc/gestionpedidos/Pedido.html
create mode 100644 doc/gestionpedidos/class-use/GestorPedidos.html
create mode 100644 doc/gestionpedidos/class-use/Main.html
create mode 100644 doc/gestionpedidos/class-use/Pedido.html
create mode 100644 doc/gestionpedidos/package-summary.html
create mode 100644 doc/gestionpedidos/package-tree.html
create mode 100644 doc/gestionpedidos/package-use.html
create mode 100644 doc/help-doc.html
create mode 100644 doc/index-files/index-1.html
create mode 100644 doc/index-files/index-2.html
create mode 100644 doc/index-files/index-3.html
create mode 100644 doc/index-files/index-4.html
create mode 100644 doc/index-files/index-5.html
create mode 100644 doc/index-files/index-6.html
create mode 100644 doc/index-files/index-7.html
create mode 100644 doc/index.html
create mode 100644 doc/Legal/ADDITIONAL_LICENSE_INFO
create mode 100644 doc/Legal/ASSEMBLY_EXCEPTION
create mode 100644 doc/Legal/LICENSE
create mode 100644 doc/Legal/jquery.md
create mode 100644 doc/Legal/jqueryUI.md
create mode 100644 doc/link.svg
create mode 100644 doc/member-search-index.js
create mode 100644 doc/module-search-index.js
create mode 100644 doc/overview-tree.html
create mode 100644 doc/package-search-index.js
create mode 100644 doc/resources/glass.png
create mode 100644 doc/resources/x.png
create mode 100644 doc/script-dir/jquery-3.7.1.min.js
create mode 100644 doc/script-dir/jquery-ui.min.css
create mode 100644 doc/script-dir/jquery-ui.min.js
create mode 100644 doc/script.js
create mode 100644 doc/search-page.js
create mode 100644 doc/search.html
create mode 100644 doc/search.js
create mode 100644 doc/stylesheet.css
create mode 100644 doc/tag-search-index.js
create mode 100644 doc/type-search-index.js
PS C:\Users\jcgsgu\GoogleDrive\segundo\Entornos de desarrollo\Tema_04> git log
commit cfd3c3224a77bb4208ace6739172e090537b (HEAD -> master)
Author: Juancho - <jcgarrido605r@fpdrioja.com>
Date: Sat Feb 21 22:42:24 2026 +0100

    Javadoc actualizado

commit e35dceb2f245bcb7fd99c3a2dd6f6570196dbf51d
Author: Juancho - <jcgarrido605r@fpdrioja.com>
Date: Sat Feb 21 22:14:07 2026 +0100

    Refactorización realizada

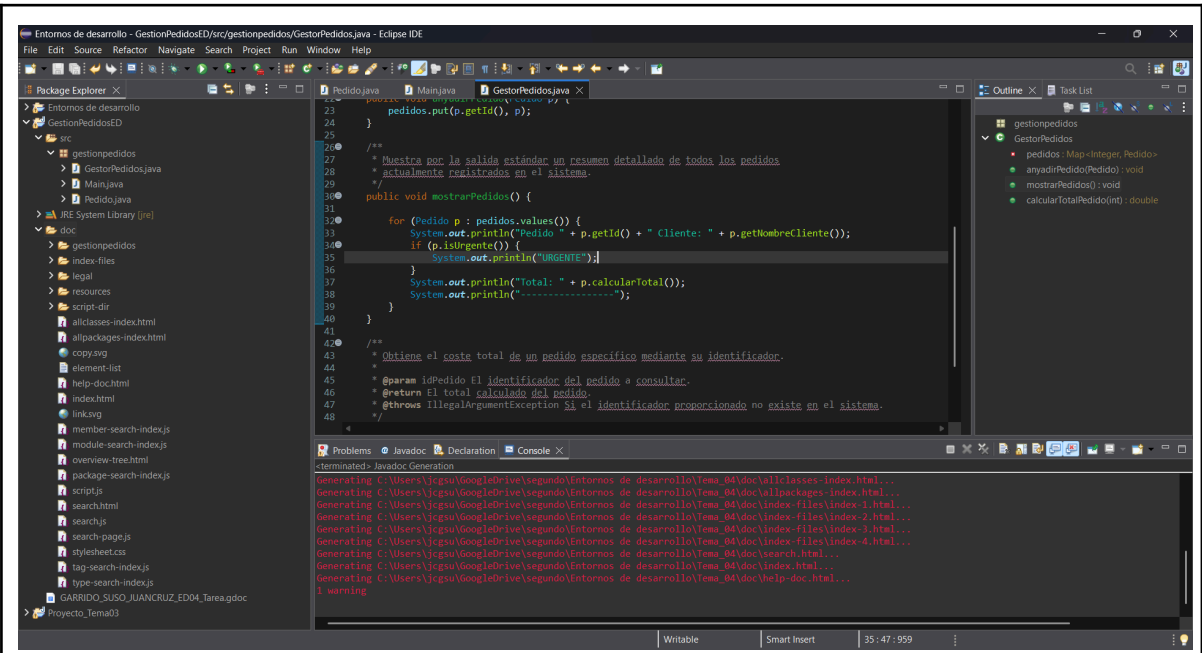
commit 0fd3284d5800b8af2f65f66a54a24bbce39a26bd
```

Git tras hacer commit sobre los cambios en la documentación del proyecto.

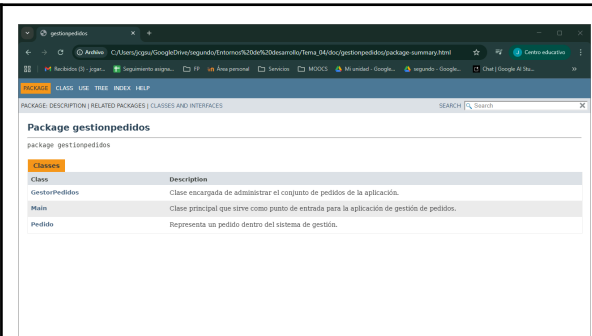
Documentación del código con Javadoc

Ejercicio 5. Documentación de clases y métodos

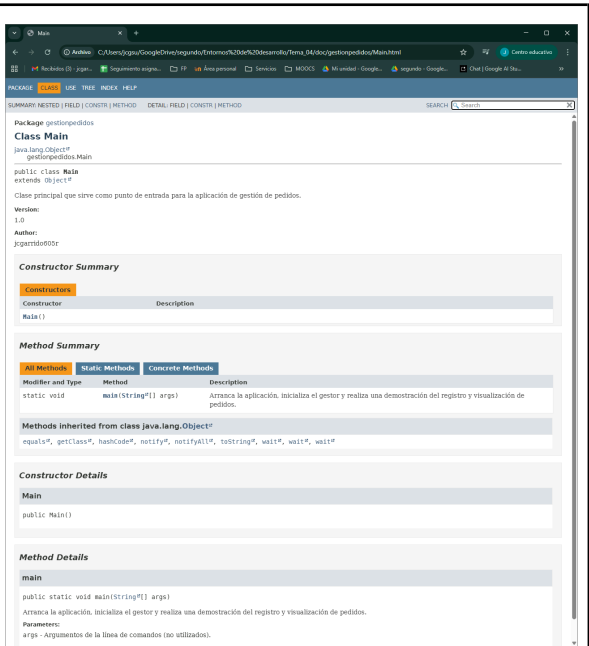
Tras generar todos los comentarios, genero el javadoc con eclipse:



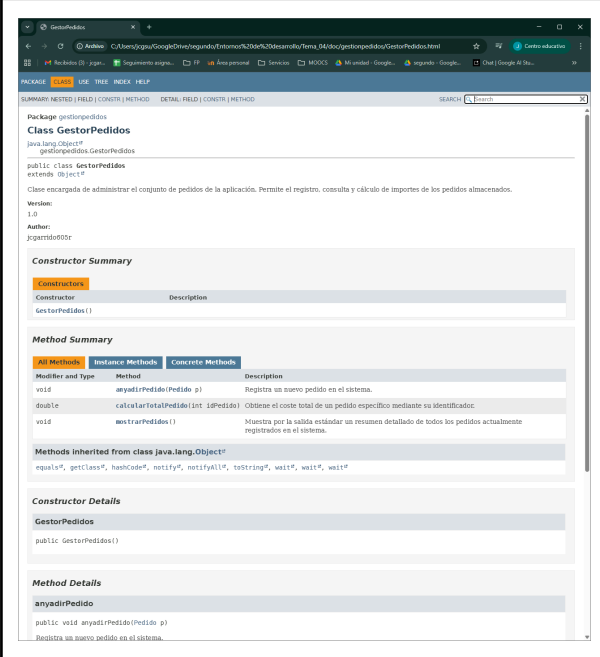
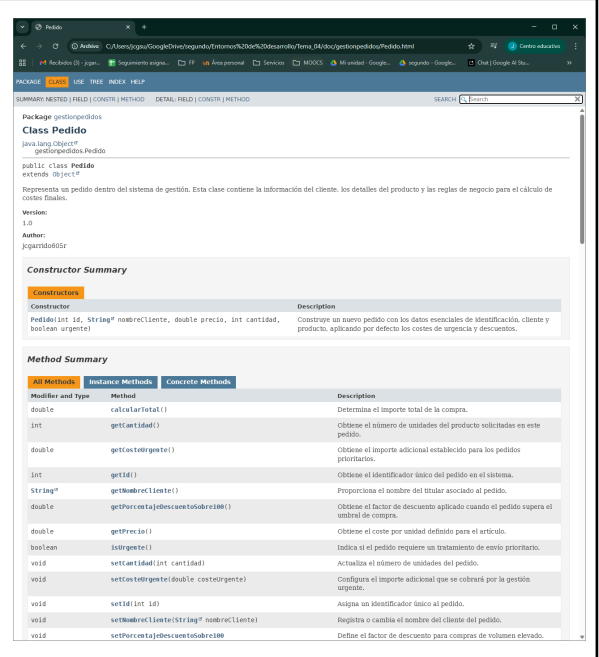
Generación exitosa



Javadoc del paquete



Del Main

	
De GestorPedidos	Y de Pedido

Incluyo ahora todo el código final del proyecto:

Main.java

```
package gestionpedidos;

/**
 * Clase principal que sirve como punto de entrada para la aplicación
 * de gestión de pedidos.
 *
 * @author jcgarrido605r
 * @version 1.0
 */
public class Main {

    /**
     * Arranca la aplicación, inicializa el gestor y realiza una demostración
     * del registro y visualización de pedidos.
     *
     * @param args Argumentos de la línea de comandos (no utilizados).
     */
    public static void main(String[] args) {

        GestorPedidos gestorPedidos = new GestorPedidos();
```

```

        Pedido pedido1 = new Pedido(1, "Laura", 30, 2, false);
        Pedido pedido2 = new Pedido(2, "Carlos", 60, 2, true);
        Pedido pedido3 = new Pedido(3, "Ana", 15, 5, false);

        gestorPedidos.anyadirPedido(pedido1);
        gestorPedidos.anyadirPedido(pedido2);
        gestorPedidos.anyadirPedido(pedido3);

        gestorPedidos.mostrarPedidos();

        System.out.println("Total pedido 2: " +
        gestorPedidos.calcularTotalPedido(2));
    }
}

```

GestorPedidos.java

```

package gestionpedidos;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Clase encargada de administrar el conjunto de pedidos de la aplicación.
 * Permite el registro, consulta y cálculo de importes de los pedidos
 * almacenados.
 *
 * @author jcgarrido605r
 * @version 1.0
 */
public class GestorPedidos {

    private Map<Integer, Pedido> pedidos = new LinkedHashMap<Integer,
    Pedido>();

    /**
     * Registra un nuevo pedido en el sistema.
     *
     * @param p El objeto Pedido que se desea añadir.
     */
    public void anyadirPedido(Pedido p) {
        pedidos.put(p.getId(), p);
    }

    /**
     * Muestra por la salida estándar un resumen detallado de todos los
     * pedidos
     * actualmente registrados en el sistema.
     */
    public void mostrarPedidos() {

```

```

        for (Pedido p : pedidos.values()) {
            System.out.println("Pedido " + p.getId() + " Cliente: " +
p.getNombreCliente());
            if (p.isUrgente()) {
                System.out.println("URGENTE");
            }
            System.out.println("Total: " + p.calcularTotal());
            System.out.println("-----");
        }
    }

    /**
     * Obtiene el coste total de un pedido específico mediante su
    identificador.
     *
     * @param idPedido El identificador del pedido a consultar.
     * @return El total calculado del pedido.
     * @throws IllegalArgumentException Si el identificador proporcionado no
    existe en el sistema.
     */
    public double calcularTotalPedido(int idPedido) {
        Pedido pedido = pedidos.get(idPedido);
        if (pedido == null) {
            throw new IllegalArgumentException("No se encuentra el pedido con
    ID: " + idPedido);
        }
        return pedido.calcularTotal();
    }
}

```

Pedido.java

```

package gestionpedidos;

/**
 * Representa un pedido dentro del sistema de gestión. Esta clase contiene la
 * información del cliente, los detalles del producto y las reglas de negocio
 * para el cálculo de costes finales.
 *
 * @author jcgarrido605r
 * @version 1.0
 */
public class Pedido {

    private int id;
    private String nombreCliente;
    private double precio;
    private int cantidad;
}

```

```

private boolean urgente;
private double costeUrgente;
private double porcentajeDescuentoSobre100;

/**
 * Construye un nuevo pedido con los datos esenciales de identificación,
 * cliente y producto, aplicando por defecto los costes de urgencia y
 descuentos.
 *
 * @param id Identificador numérico único del pedido.
 * @param nombreCliente Nombre del cliente que realiza la compra.
 * @param precio Valor unitario del artículo.
 * @param cantidad Número de unidades solicitadas.
 * @param urgente Estado que define si el pedido tiene prioridad de envío.
 */
public Pedido(int id, String nombreCliente, double precio, int cantidad,
boolean urgente) {
    this.setId(id);
    this.setNombreCliente(nombreCliente);
    this.setPrecio(precio);
    this.setCantidad(cantidad);
    this.setUrgente(urgente);
    this.setCosteUrgente(20);
    this.setPorcentajeDescuentoSobre100(0.1);
}

/**
 * Determina el importe total de la compra. El cálculo final incluye
 * recargos por gestión de urgencia y beneficios de descuento por volumen
 * de compra si se supera el límite establecido.
 *
 * @return El importe total neto que el cliente debe abonar.
 */
public double calcularTotal() {
    double total = getPrecio() * getCantidad();
    if (isUrgente()) {
        total = total + getCosteUrgente();
    }
    if (total > 100) {
        total = total - (total * getPorcentajeDescuentoSobre100());
    }
    return total;
}

/**
 * Indica si el pedido requiere un tratamiento de envío prioritario.
 * @return true si el pedido es urgente, false en caso contrario.
 */
public boolean isUrgente() {
    return urgente;
}

```

```

/**
 * Modifica la prioridad de envío del pedido.
 * @param urgente true para marcarlo como prioritario.
 */
public void setUrgente(boolean urgente) {
    this.urgente = urgente;
}

/**
 * Obtiene el número de unidades del producto solicitadas en este pedido.
 * @return Cantidad de artículos.
 */
public int getCantidad() {
    return cantidad;
}

/**
 * Actualiza el número de unidades del pedido.
 * @param cantidad Nueva cantidad de artículos.
 */
public void setCantidad(int cantidad) {
    this.cantidad = cantidad;
}

/**
 * Obtiene el coste por unidad definido para el artículo.
 * @return Precio unitario.
 */
public double getPrecio() {
    return precio;
}

/**
 * Establece el coste por unidad del artículo.
 * @param precio Nuevo precio unitario.
 */
public void setPrecio(double precio) {
    this.precio = precio;
}

/**
 * Proporciona el nombre del titular asociado al pedido.
 * @return Nombre del cliente.
 */
public String getNombreCliente() {
    return nombreCliente;
}

/**
 * Registra o cambia el nombre del cliente del pedido.

```

```

        * @param nombreCliente Nombre del titular.
        */
    public void setNombreCliente(String nombreCliente) {
        this.nombreCliente = nombreCliente;
    }

    /**
     * Obtiene el identificador único del pedido en el sistema.
     * @return ID del pedido.
     */
    public int getId() {
        return id;
    }

    /**
     * Asigna un identificador único al pedido.
     * @param id Nuevo identificador.
     */
    public void setId(int id) {
        this.id = id;
    }

    /**
     * Obtiene el factor de descuento aplicado cuando el pedido supera el
    umbral de compra.
     * @return Valor decimal del porcentaje de descuento.
     */
    public double getPorcentajeDescuentoSobre100() {
        return porcentajeDescuentoSobre100;
    }

    /**
     * Define el factor de descuento para compras de volumen elevado.
     * @param porcentajedescontoSobre100 Nuevo porcentaje en formato decimal
    (ej: 0.1 para 10%).
     */
    public void setPorcentajeDescuentoSobre100(double
    porcentajedescontoSobre100) {
        this.porcentajeDescuentoSobre100 = porcentajedescontoSobre100;
    }

    /**
     * Obtiene el importe adicional establecido para los pedidos
    prioritarios.
     * @return Valor del coste por urgencia.
     */
    public double getCosteUrgente() {
        return costeUrgente;
    }

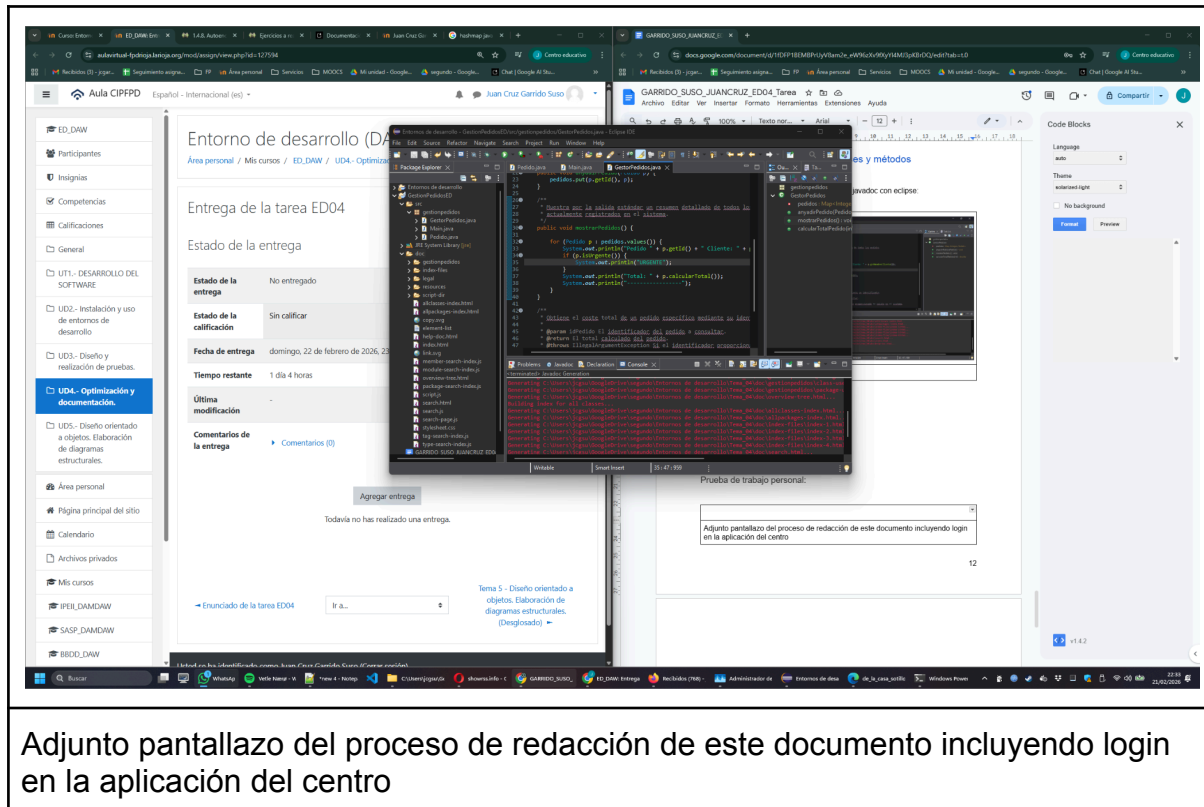
    /**

```

```
    * Configura el importe adicional que se cobrará por la gestión urgente.  
    * @param costeUrgente Nuevo importe de recargo.  
    */  
    public void setCosteUrgente(double costeUrgente) {  
        this.costeUrgente = costeUrgente;  
    }  
}
```

Anexo

Prueba de trabajo personal:



Adjunto pantallazo del proceso de redacción de este documento incluyendo login en la aplicación del centro